

HelixVPN — Phase 2 Build Specification (Parity + Reach)

HelixVPN — Phase 2 Build Specification (Parity + Reach)

Revision: 2 **Last modified:** 2026-07-04T12:00:00Z **Status:** primary research source — elaborated + made authoritative by [../final/08-phase2-parity-wbs.md](#) (the executable WBS with HVPN-P2-NNN work items, the 8 P2-AC + 5 P2-SLO release gates, and captured-evidence requirements). Where this document and [final/08-phase2-parity-wbs.md](#) disagree, **final/ wins** per SPECIFICATION.md §12 versioning discipline — this document is preserved as the cited primary source ([04_P2] throughout [final/](#)), not re-derived.

Companion to: [HelixVPN-Architecture-Refined.md](#), [HelixVPN-Phase0-Spike.md](#), [HelixVPN-Phase1-MVP.md](#). **Entry condition:** Phase 1 MVP shipped — self-host from zero, enroll/advertise/policy/reconcile all working, auto transport ladder with plain/LWO/MASQUE, no-logging verified. **Goal of Phase 2:** reach **full Mullvad feature parity and surpass it on reach** — the complete obfuscation set, traffic-analysis defense, *direct* peer-to-peer paths (stop always relaying), multi-hop, a post-quantum handshake, desktop apps, policy-as-code/GitOps, and a highly-available multi-region gateway fleet.

The discipline of Phase 2: **everything is additive**. Phase 1 drew the seams (TransportPolicy.order, Peer.endpoint, federated coordinator, PKI hooks) precisely so none of the below requires reshaping an existing interface.

0. Scope delta (Phase 1 → Phase 2)

Concern	Phase 1 (MVP)	Phase 2 (Parity + Reach)
Transports	plain-UDP, LWO, MASQUE	

Concern	Phase 1 (MVP)	Phase 2 (Parity + Reach)
Traffic analysis	none	+ Shadowsocks-wrap, UDP-over-TCP ; refined auto-ladder DAITA — constant packet sizing + cover traffic (maybenot)
Data path	always relay via gateway (hub-spoke)	direct peer-to-peer with NAT traversal; relay fallback only
Topology	single hop	multi-hop (entry/exit separation)
Crypto	WireGuard	+ post-quantum PSK (harvest-now-decrypt-later resistance)
Apps	iOS/Android/Linux + Connector + Console	+ Windows/macOS desktop apps
Policy ops	Console/CLI apply	policy-as-code + GitOps pipeline
Availability	single gateway, single control plane	HA control plane + multi-region gateway fleet + failover
Events	single Redis	NATS JetStream federation across regions

Out of scope (→ Phase 3): HarmonyOS/Aurora builds, the WASM web tunnel, billing, formal security audit + reproducible builds.

1. The full transport / obfuscation set

Phase 1 proved the Transport trait with three impls. Phase 2 fills out the matrix — **all new transports are just more impls of the same trait** (architecture §3.2), shared byte-for-byte across client, connector, and edge.

1.1 Shadowsocks-wrap

WireGuard datagrams wrapped in a Shadowsocks AEAD stream — the canonical "looks like random/TLS-ish TCP" evasion for China-style DPI where even QUIC is throttled.

```
// helix-transport/src/shadowsocks.rs (sketch)
pub struct ShadowsocksTransport {
```

```

    // AEAD cipher (chacha20-poly1305 / aes-256-gcm), session keys
    // derived from
    // the pre-shared transport password (separate from WG keys)
    stream: shadowsocks_crypto::AeadStream,
}
// send: frame WG datagram with 2-byte length prefix -> AEAD
// encrypt -> TCP
// recv: AEAD decrypt -> deframe -> WG datagram

```

Reuse shadowsocks-rust crypto primitives rather than re-implementing AEAD framing. Carries the same loss caveat as UoT (it's over TCP), so the ladder prefers it only when UDP/QUIC are blocked.

1.2 UDP-over-TCP (UoT)

Last-resort transport when **all** UDP is blocked. WG datagrams length-prefixed over a single TCP connection. Accept the head-of-line-blocking penalty; it exists purely to keep a tunnel *possible* on the most hostile networks (matches Mullvad's `udp2tcp`).

1.3 LWO refinement

Phase 1 shipped a basic XOR/padding LWO. Phase 2 hardens it into a proper lightweight scheme: per-session keyed obfuscation of the WG header bytes that DPI signatures key on (the message-type/reserved fields), plus randomized padding — cheap evasion of *naive* WG fingerprinting without QUIC's overhead. (Mullvad's "LWO" is the design reference.)

1.4 The auto-escalation ladder (refined)

The client walks `TransportPolicy.order` on repeated handshake failure, now with the full set and **regional priors** pushed from the coordinator:

```

default order (unrestricted): [plain-udp]
restricted prior (e.g. region): [plain-udp, lwo, masque-h3,
shadowsocks, udp-over-tcp]

```

- Each step has a failure budget (N handshakes / T seconds) before escalating.
- On success, the working transport is **remembered per-network** (SSID/gateway fingerprint) so reconnects on the same hostile network skip straight to what worked — this is the UX difference between "VPN that sometimes connects" and "VPN that just works."
- The coordinator can push a **regional prior** (e.g., "clients connecting from CN-resolved IPs start at shadowsocks") via `TransportPolicy.order`, so users in censored regions don't pay the escalation latency every time.

- Telemetry records **only** "transport X succeeded after N escalations in region R" (aggregate, no per-user data) → feeds the Censorship-Evasion Success dashboard (architecture §9).
-

2. DAITA — Defense Against AI-guided Traffic Analysis

Even fully encrypted, packet **size/timing/frequency** patterns can fingerprint which site a user visits. DAITA defeats this. **Do not roll your own** — adopt the **maybenot** framework (the same one behind Mullvad's DAITA): a lightweight state-machine engine for padding + cover-traffic "machines."

2.1 What it does

1. **Constant packet sizing** — pad WG datagrams to uniform sizes so size-based fingerprinting fails.
2. **Cover traffic** — inject dummy packets to mask real activity bursts.
3. **Timing normalization** — break the correlation between application events and on-wire timing.

2.2 Where it sits

DAITA shaping is a layer **above WireGuard, below the transport** — it operates on the stream of WG datagrams (padding/injecting) before they hit the obfuscating transport. It's orthogonal to *which* transport is in use (works with plain-UDP or MASQUE alike).

TUN → WG encrypt → [DAITA shaping: pad + inject cover] → Transport → wire

2.3 Integration

```
// helix-core: a shaping stage driven by maybenot machines
pub struct Daita {
    framework: maybenot::Framework,    // runs padding/blocking
    "machines"
    // machines are config (downloaded as part of NetworkMap), not
    code
}
impl Daita {
    // called for each outgoing WG datagram; may pad it and/or
    schedule cover packets
    fn on_packet(&mut self, dg: &mut Bytes, now: Instant) ->
        Vec<ScheduledAction> { /* ... */ }
    // timer-driven: emit scheduled cover/padding packets
}
```

```
fn tick(&mut self, now: Instant) -> Vec<Bytes> { /* ... */ }
}
```

- **Machines as data:** padding machines are distributed via the NetworkMap (a new daita field), so defenses can be tuned/updated server-side without shipping a client build. Toggle per the architecture's parity matrix.
 - **Cost is real:** DAITA trades bandwidth/latency for privacy. It's **off by default, opt-in** (Mullvad's stance) and surfaced in the Access app as "maximum privacy" with an honest cost note.
 - **Correctness caution (carried from Phase 0 risk table):** traffic-analysis defense is subtle; lean on maybenot's vetted machines rather than inventing schemes. Validate with a closed-world website-fingerprinting test harness before claiming the feature.
-

3. Direct peer-to-peer with NAT traversal (the big networking upgrade)

Phase 1 relayed **all** peer traffic through the gateway (simple, matches the proven slice). Phase 2 makes the gateway a **coordinator + relay-of-last-resort**, with traffic going **directly** between client and connector wherever NAT allows — lower latency, far less gateway bandwidth, better scaling. This is the Tailscale/WireGuard roaming model.

3.1 Endpoint discovery (STUN-like)

Each node learns its own public-facing ip:port candidates:

- **Local candidates:** all local interface addresses.
- **Server-reflexive candidates:** discovered by sending a probe to the gateway's STUN-like endpoint, which echoes the observed src ip:port (reveals the NAT mapping).
- Candidates are reported to the coordinator via ReportStatus and distributed to authorized peers inside Peer.endpoint candidate lists (the field Phase 1 reserved).

3.2 Hole punching

client and connector each know the other's candidate list (via coordinator signaling)

both simultaneously send WG handshake probes to all of the peer's candidates

→ for full-cone / restricted-cone / port-restricted NATs: a path opens

→ WireGuard's roaming picks the working endpoint automatically (it just works once a packet arrives)

symmetric NAT on both ends → hole punching fails → fall back to relay (§3.4)

WireGuard's design helps here: it has no client/server distinction at the protocol level and **roams to whatever source address a valid handshake arrives from**, so once a hole is punched, WG latches on with no extra machinery.

3.3 The coordinator as signaling server

The existing WatchNetworkMap stream **is** the signaling channel — no new service. When two authorized peers come online, the coordinator pushes each the other's candidate list as a Peer.endpoint delta; both begin hole punching. This reuses Phase 1's stream rather than adding a separate signal server (NetBird-style designs use a dedicated signal service; HelixVPN folds it into the coordinator).

3.4 Relay fallback (DERP-style)

When direct fails (symmetric NAT, hostile firewall), traffic relays through the gateway over an **encrypted relay** — exactly the Phase 1 data path, now the *fallback* rather than the default. Name it helix-relay (a mode of the edge). The relay sees only encrypted WG datagrams keyed by destination public key; it cannot read traffic (it's below the WG crypto boundary). This preserves no-logging even on the relay path.

3.5 Path selection & upgrade

- Start on relay (instant connectivity), **attempt direct in the background**, upgrade seamlessly when a direct path is confirmed (latency drop, no user-visible reconnect).
 - Continuously health-check the direct path; **downgrade to relay** if it degrades. Report current path (direct/relay) in status for the UI.
-

4. Multi-hop (entry/exit separation)

Generalizes the architecture's §3.5. A client routes Client → Entry → Exit → {internet | connector} so no single node sees both who you are and where you're going.

4.1 Mechanism — nested WireGuard

Client holds TWO WG sessions:

outer: Client ↔ Entry (entry sees client IP, not destination)

inner: Client ↔ Exit (exit sees destination, not client IP – only sees Entry)
packets: WG_inner(payload) then WG_outer(WG_inner(...)) →
Entry decapsulates outer → forwards to Exit

- Per-hop keys, distributed via the NetworkMap (a hops list the coordinator computes from policy/user choice).
- Entry and Exit can be different gateways in **different regions/jurisdictions** (the privacy point).
- Transport/obfuscation applies to the **outer** session (Client↔Entry); the inner ride is plain WG inside the outer tunnel.

4.2 Orchestration

The coordinator computes the hop chain (respecting exitNodes policy and user selection), assigns per-hop keys, and pushes a multi-hop NetworkMap. The client core builds the nested sessions. No new wire protocol — multi-hop is layered WG + map fields.

5. Post-quantum handshake

WireGuard's Curve25519 handshake is vulnerable to **harvest-now-decrypt-later**: an adversary records traffic today and decrypts it once quantum computers mature. Phase 2 closes this the way Mullvad does — **without forking WireGuard's crypto** — by injecting a post-quantum-derived **pre-shared key (PSK)** into the WG handshake.

5.1 Approach: PQ-derived PSK

WireGuard already supports an optional symmetric PSK mixed into the handshake. If that PSK is established via a **post-quantum KEM**, the session gains PQ resistance even though the core handshake stays classical (defense-in-depth: an attacker must break *both*).

on session setup, over the (already-authenticated) control channel:

```
client → gateway: PQ-KEM public key (ML-KEM / Kyber, FIPS 203)
gateway → client: KEM ciphertext
both derive:      shared secret → HKDF → WG PSK
WG handshake then proceeds with that PSK mixed in
rotate PSK on each rekey interval.
```

- **KEM choice:** ML-KEM (FIPS 203, the standardized Kyber) as the primary; optionally combine with a code-based KEM (Classic McEliece) for hybrid conservatism — Mullvad's

original PQ tunnels combined Classic McEliece + Kyber for exactly this hedge. Keep it pluggable in pki + helix-core.

- **Hybrid, never PQ-only:** always combine PQ with the classical exchange so a flaw in the (younger) PQ primitive can't *weaken* you below today's security.
- **Where it runs:** the KEM exchange rides the existing authenticated control channel (Coordinator RPC), so no new public listener and no unauthenticated PQ endpoint to attack.
- **Alternative to evaluate:** the open **Rosenpass** protocol (a separate PQ key-exchange daemon that feeds WG a PSK) — viable if you prefer an audited external protocol over an in-house KEM exchange. Decide during Phase 2 spike.

5.2 Cost & UX

PQ keys are large (ML-KEM ciphertexts are KBs; Classic McEliece public keys are large) — so the exchange happens at **session setup and rekey**, not per packet; steady-state cost is zero. Surface as a "Quantum-resistant" toggle, on by default for new tunnels where both ends support it (negotiated capability).

6. Desktop apps (Windows, macOS)

The Flutter UI + Rust core already build for desktop (proven in Phase 0/1 on Linux); Phase 2 adds the two remaining first-class desktop tunnel shims (architecture §5.3).

Platform	Tunnel mechanism	Shim notes
Windows	wireguard-nt / wintun driver	small Windows service runs the privileged tunnel; Flutter app talks to it over a local IPC (named pipe); service hosts helix-core. Code-sign the driver + service.
macOS	NEPacketTunnelProvider (System/Network Extension)	same Network Extension model as iOS but desktop-class memory (the §6 iOS ceiling does not bite here); notarize + sign.

Everything above the shim — UI, settings, account, transport selection, kill-switch, split-tunnel, multi-hop picker, DAITA toggle — is the **same helix-ui + helix-core** as every other platform. The

desktop split-tunnel (per-app routing) is the main net-new platform-specific surface (Windows WFP filters; macOS uses app-bound rules).

7. Policy-as-code & GitOps

Phase 1 applied policy via Console/CLI. Phase 2 makes **Git the source of truth** (the architecture's GitOps model, and a natural fit with your existing Helix-ecosystem workflow).

7.1 Repo + pipeline

```
helixvpn-policy/                                # per-tenant (or per-org)
Git repo
├─ policy.jsonc                                # the ACL document (Phase 1
§7.1 format)
├─ networks.jsonc                              # host/CIDR ↔ connector
mappings
└─ .github/workflows/apply.yml                 # CI: validate → dry-run
compile → apply on merge

PR opened          → CI: schema-validate + `helixvpnctl policy
compile --dry-run`
                  → posts the *diff of effects* (which devices
gain/lose which access)
PR merged          → CI: `helixvpnctl policy apply` (authenticated
via tenant API token)
                  → control plane: persist new version, emit
policy.compiled
                  → coordinator: push deltas (the Phase 1 §10
flow), < 1s convergence
```

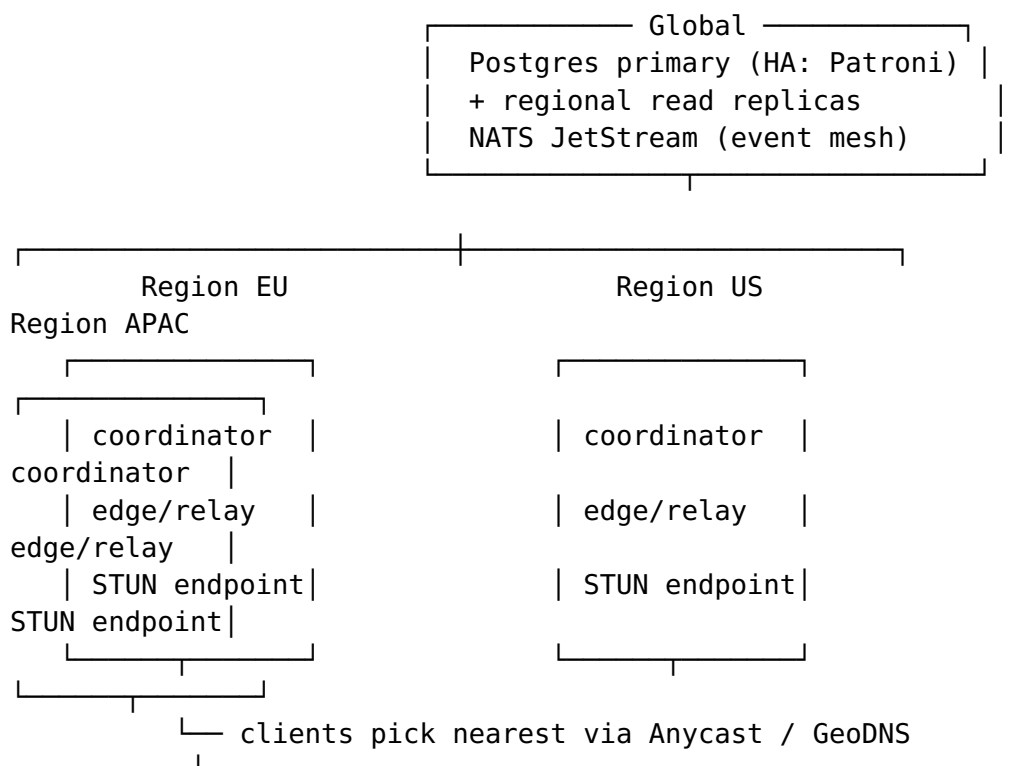
7.2 Why the "diff of effects" matters

A one-line ACL change can silently grant a contractor access to a camera network. CI rendering the **compiled effect delta** (not just the text diff) on every PR is the safety rail — review the *consequence*, not the syntax. Secrets (WG keys) never live in this repo; only declarative intent. Encrypt any sensitive references with sops/age (carried from the original doc's GitOps section).

8. High availability & multi-region

Phase 1 was a single VPS. Phase 2 is a **fleet**: HA control plane, gateways in multiple regions, automatic failover. The same images — only topology changes (architecture §10).

8.1 Topology



8.2 Control plane HA

- **Coordinators are stateless** (topology graph is a rebuildable cache hydrated from Postgres + NATS). Run N per region behind a load balancer; an agent's WatchNetworkMap stream can be served by any coordinator, and re-pins to another on failure (resume via known_version).
- **Postgres**: single logical primary with synchronous replicas + automated failover (Patroni); regional read replicas serve coordinator hydration. Control-plane *writes* are low-volume (enroll, policy, advertise) so a single primary scales fine.
- **NATS JetStream** replaces single-Redis as the cross-region event mesh: events published in any region propagate to all coordinators (durable subjects mirror the Phase 1 stream taxonomy 1:1 — the swap is transport-only, as promised).

8.3 Gateway/edge fleet & client steering

- **Steering:** Anycast IP (single address, routed to nearest edge) is ideal for UDP/QUIC; GeoDNS is the simpler fallback. Clients also receive a **candidate gateway list** in the NetworkMap and pick by measured RTT.
- **Floating IPs** per region for fast in-region failover (carried from the original doc's DigitalOcean pattern).
- **Relay reachability:** because relay traffic is encrypted WG keyed by destination pubkey, any region's relay can forward for any tenant — relays are fungible.

8.4 Failover flow

```
health probe: EU edge unreachable → emit
gateway.failover{from:eu-1, to:eu-2}
coordinator: push NetworkMap delta (new gateway endpoint) to
affected clients
clients: re-handshake to eu-2 (transport + PSK re-established);
existing direct P2P paths unaffected
  — target: client reconnection < 3s; direct-path sessions
survive untouched —
```

Existing **direct peer-to-peer** sessions don't depend on the gateway, so a gateway failure only affects relay-path users and control-channel freshness — a strong availability property that falls out of §3.

9. Observability additions

Extend the Phase 1 / architecture §9 stack for the new surface:

- **Transport mix** by region (how often each obfuscation tier is used) and **escalation depth** (censorship pressure signal).
 - **Direct-vs-relay ratio** (the scaling KPI — high relay ratio = bandwidth cost + a NAT-traversal problem to investigate).
 - **Hole-punch success rate** by NAT-type bucket.
 - **PQ handshake adoption** (% sessions PQ-protected).
 - **DAITA overhead** (cover-traffic bytes as % of real) — so the privacy/cost tradeoff is visible.
 - **Multi-region:** per-region coordinator health, NATS consumer lag, Postgres replication lag, cross-region map-convergence p99.
 - Privacy invariant holds everywhere: still **counts and health only, never destinations/flows**.
-

10. Testing & acceptance

10.1 New test surfaces

- **NAT-traversal matrix:** automated tests across NAT-type combinations (full-cone, restricted, port-restricted, symmetric) using netns + simulated NATs; assert direct-path success where theoretically possible and clean relay fallback where not.
- **Obfuscation conformance:** each transport passes through a DPI-simulation gauntlet (block UDP, block QUIC, block non-TLS) and a tshark classifier confirms the intended disguise.
- **DAITA efficacy:** a closed-world website-fingerprinting harness must show classifier accuracy drop materially with DAITA on (the actual privacy claim — measure it, don't assert it).
- **PQ interop:** classical-only ↔ PQ-capable negotiation; downgrade safety (a PQ failure must never silently drop to *weaker-than-classical*).
- **Multi-hop:** verify entry cannot observe destination and exit cannot observe client IP (packet-capture assertions at each hop).
- **HA/chaos:** kill a coordinator, a Postgres primary, a whole region; assert stream re-pin, failover timings, and zero policy/identity loss.

10.2 Phase 2 definition of done

1. All transports present; auto-ladder + per-network memory + regional priors working through a hostile-network simulation.
2. DAITA toggle measurably reduces fingerprinting in the test harness; cost surfaced honestly in-app.
3. Direct P2P established between client and connector behind common NATs; seamless relay fallback; live path indicator.
4. Multi-hop entry/exit verified jurisdiction-separable with the privacy invariants proven by capture.
5. PQ handshake on by default where supported; hybrid (never PQ-only); zero steady-state cost.
6. Windows + macOS desktop apps reach parity with Linux (incl. split-tunnel).
7. GitOps: a merged policy PR converges the fleet < 1 s with a reviewed effect-diff.
8. Survive a regional outage with client reconnection < 3 s and no control-plane data loss.

10.3 SLOs added

SLO	Target
	> 90% within 10 s

SLO	Target
direct-path establishment (non-symmetric NAT)	
relay→direct upgrade (when possible)	< 15 s, no user-visible reconnect
gateway failover → client reconnected	p99 < 3 s
cross-region event propagation	p99 < 2 s
PQ session adoption (capable pairs)	> 95%

11. What graduates to Phase 3

Phase 3 is the **full-reach + hardening** phase: HarmonyOS NEXT and Aurora OS builds (forked Flutter + native tunnel shims under the unchanged UI/core), the WASM browser-scoped MASQUE proxy for the web Console, multi-tenant billing-optional, a third-party **security audit**, and **reproducible builds** (so users can verify the binaries match the source — table stakes for a privacy product). Again additive: Phase 2's transport trait, map protocol, coordinator federation, and shim pattern already accommodate all of it.

End of Phase 2 build specification. Quartet complete: Architecture (what) → Phase 0 (prove the hard parts) → Phase 1 (ship a self-hostable MVP) → Phase 2 (Mullvad parity + multi-region reach). Each phase is the strict, additive evolution of the last; no interface defined earlier is reshaped later, by design.